



محاسبه کانولوشن ماتریس ها با UART

درس آزمایشگاه معماری کامپیوتر

استاد : سرکار خانم دکتر علی اعظم

دانشجویان : دیانا سنابادی

امیر محمد امیرایی

پارسا عربشاهی

مقدمه

کانولوشن یکی از مهم‌ترین عملیات‌ها برای پردازش سیگنال و تصویر است در این پروژه کانولوشن دو ماتریس روی سخت‌افزار FPGA پیاده‌سازی شود به این صورت که داده‌ها از طریق رابط UART از کامپیوتر به FPGA ارسال می‌شوند الگوریتم‌های پردازشی روی سخت‌افزار و کار با پروتکل UART و مدیریت داده‌ها و طراحی سیستم‌های مبتنی بر FSM یا همان ماشین حالت محدود پیاده‌سازی می‌شود

پروژه شامل سه مرحله دریافت داده‌ها سپس پردازش آن‌ها برای محاسبه کانولوشن و در نهایت ارسال نتایج به کامپیوتر برای مشاهده خروجی می‌باشد

طراحی الگوریتم

ابتدا الگوریتم را به سه بخش تقسیم می‌کنیم و هر کدام را جداگانه می‌نویسیم و در آخر بهم وصل می‌کنیم

(1) دریافت داده‌ها از کامپیوتر UART RX

(2) محاسبه کانولوشن دو ماتریس

(3) ارسال نتایج به کامپیوتر UART TX

توضیحات :

(1) دریافت داده‌ها از کامپیوتر UART RX

برای دریافت داده‌ها از رابط UART یک ماشین حالت طراحی شده است این ماشین حالت مسئول خواندن اندازه ماتریس‌ها m و n و سپس دریافت عناصر ماتریس A و B است ابتدا کلاک ۴۰ مگاهرتز FPGA با استفاده از تقسیم‌کننده به یک سیگنال tick در نرخ ۱۱۵۲۰۰ baud تبدیل می‌شود تا بتواند با نرخ استاندارد UART داده‌ها را نمونه‌برداری کند ماشین حالت UART شامل سه مرحله اصلی است IDLE منتظر بیت شروع Start Bit می‌ماند DATA هشت بیت داده را به ترتیب LSB first دریافت می‌کند STOP بیت توقف را بررسی می‌کند و در صورت صحت سیگنال دریافت موفق rx done فعال می‌شود

یک FSM سطح بالاتر برای دریافت ماتریس‌ها طراحی شده است

GET N دریافت اندازه ماتریس A

GET M دریافت اندازه ماتریس B

READ A دریافت عناصر ماتریس A و ذخیره در یک آرایه دو بعدی داخلی

READ B دریافت عناصر ماتریس B و ذخیره آن

FINISH آماده‌سازی برای محاسبه کانولوشن

هر عنصر ماتریس در بافر داخلی با بیت‌بندی مناسب هشت بیت برای هر عدد ذخیره می‌شود تا پردازش سخت‌افزاری ساده و سریع انجام شود

(2) محاسبه کانولوشن

پس از دریافت کامل داده‌ها الگوریتم کانولوشن اجرا می‌شود کانولوشن به صورت بدون padding و با اندازه خروجی $n \times m$ انجام می‌شود برای هر مکان r و c در ماتریس خروجی مجموع حاصل ضرب عناصر متناظر ماتریس A و B محاسبه می‌شود

$$C(r,c) = \sum_{kr=0}^{m-1} \sum_{kc=0}^{m-1} A(r+kr, c+kc) \times B(kr, kc)$$

Scanned with CamScanner

با استفاده از حلقه‌های تو در تو این عملیات انجام شده و نتایج در بافر خروجی ۳۲۰۰ بیتی ذخیره می‌شوند هر نتیجه ۳۲ بیت است و 10×10 خانه برای ذخیره نتایج در نظر گرفته شده است

(3) ارسال نتایج به کامپیوتر UART TX

پس از محاسبه کانولوشن نتایج به کامپیوتر ارسال می‌شوند تا توسط نرم‌افزاری مانند CoolTerm قابل مشاهده باشند یک ماشین حالت برای UART TX طراحی شده که هر عدد ۳۲ بیتی را به رشته‌های ASCII ده‌دهی تبدیل می‌کند سپس داده‌ها به صورت بایت به بایت و با بیت شروع و توقف ارسال می‌شوند هر عدد با CRLF پایان می‌یابد تا در نرم‌افزار مقصد به صورت خط به خط قابل نمایش باشد این طراحی نشان‌دهنده توجه به قابلیت مشاهده نتایج و ارتباط صحیح سخت‌افزار با نرم‌افزار است

شرح کد بخش rx :

```
;tick <= '0  
if cnt = DIV-1 then  
;cnt <= 0  
;tick <= '1  
else  
;cnt <= cnt + 1  
;end if
```

این قسمت سیگنال tick را تولید می کند که زمان نمونه برداری UART را مشخص می کند

کلاک اصلی مدار 40 مگاهرتز است اما UART با سرعت 115200 بیت بر ثانیه کار می کند
برای هماهنگ شدن با UART کلاک تقسیم می شود
ثابت DIV برابر 174 است یعنی هر 174 سیکل کلاک یک tick تولید می شود

در هر سیکل ابتدا tick صفر می شود

شمارنده cnt افزایش پیدا می کند

وقتی cnt به مقدار نهایی برسد شمارنده صفر می شود و tick یک می شود

tick در واقع یک سیگنال enable است که مشخص می کند چه زمانی باید یک بیت UART خوانده شود

```
;rx_done <= '0
```

این خط در هر سیکل کلاک سیگنال rx_done را صفر می کند

rx_done فقط زمانی یک می شود که یک بایت کامل با موفقیت دریافت شده باشد

پس در شروع هر سیکل صفر می شود تا فقط یک پالس کوتاه ایجاد کند

```
if tick='1' then
```

ماشین حالت گیرنده فقط زمانی اجرا می شود که tick فعال باشد

به این ترتیب نمونه برداری دقیقاً با سرعت UART انجام می شود و از خواندن تصادفی جلوگیری می شود

```
case rx_state is
```

```
<= when IDLE
```

```
if rx='0' then
```

```
;bit_i <= 0
```

```
;rx_state <= DATA
```

```
;end if
```

در حالت IDLE خط UART در وضعیت عادی قرار دارد که مقدار آن یک است
اگر مقدار rx صفر شود یعنی بیت شروع دریافت شده است در این لحظه شمارنده بیت ها صفر می شود و ماشین حالت وارد وضعیت DATA می شود از این لحظه است که خواندن داده آغاز می شود

```
<= when DATA
```

```
rx_byte(bit_i) <= rx; -- LSB first
```

در این حالت هر بار که tick فعال می شود یک بیت خوانده می شود
بیت خوانده شده داخل rx_byte ذخیره می شود سپس داده ها به صورت LSB first دریافت می شوند یعنی بیت کم ارزش زودتر ذخیره می شود

```
if bit_i = 7 then
```

```
;rx_state <= STOP
```

```
else
```

```
;bit_i <= bit_i + 1
```

```
;end if
```

وقتی هشت بیت کامل دریافت شد ماشین حالت وارد وضعیت STOP می شود در غیر این صورت شمارنده بیت افزایش پیدا می کند تا بیت بعدی خوانده شود

```
<= when STOP
```

```
if rx='1' then
```

```
;rx_done <= '1
```

```
;end if
```

```
;rx_state <= IDLE
```

```
;end case
```

```
;end if
```

در این مرحله بیت توقف بررسی می شود

اگر مقدار rx برابر یک باشد یعنی فریم UART درست دریافت شده است و rx_done یک می شود تا اعلام کند یک بایت کامل آماده است بعد از آن ماشین حالت دوباره به IDLE برمی گردد تا بایت بعدی دریافت شود

>> حالا بعد از اینکه یک بایت کامل دریافت شد باید مشخص شود که این بایت مربوط به چه مرحله ای است

این کار با ماشین حالت سطح بالاتر انجام می شود<<

```
if rx_done='1' and stage /= FINISH then
```

این بخش فقط زمانی اجرا می شود که یک بایت کامل دریافت شده باشد

اگر سیستم به حالت FINISH رسیده باشد دیگر داده ای دریافت نمی شود

```
case stage is
```

```
<= when GET_N
```

```
;tmp := to_integer(unsigned(rx_byte))
```

بایت دریافتی به عدد تبدیل می شود

```
if tmp >= 1 and tmp <= 10 then
```

اندازه ماتریس بررسی می شود تا در محدوده مجاز باشد

```
;n <= tmp
```

```
;totalA <= tmp * tmp
```

```
;cntA <= 0
```

```
;bufA <= (others => (others => '0'))
```

```
;stage <= GET_M
```

```
;conv_done <= '0
```

```
;Cbuf <= (others => '0')
```

```
;outN_sig <= (others => '0')
```

```
;end if
```

```
<= when GET_M
```

اگر مجاز باشد اندازه ماتریس اول ذخیره می شود

تعداد کل عناصر ماتریس محاسبه می شود

شمارنده صفر می شود

بافر پاک می شود

ماشین حالت وارد مرحله دریافت m می شود

همزمان خروجی های قبلی نیز پاک می شوند تا محاسبه جدید از صفر شروع شود

```
;tmp := to_integer(unsigned(rx_byte))
```

```
if tmp >= 1 and tmp <= 10 then
```

```
;m   <= tmp
```

```
;totalB <= tmp * tmp
```

```
;cntB  <= 0
```

```
;bufB  <= (others => (others => '0'))
```

```
;stage <= READ_A
```

```
;end if
```

این بخش دقیقاً مشابه GET_N است اما برای ماتریس دوم اجرا می شود

```
<= when READ_A
```

بعد از دریافت m سیستم وارد مرحله خواندن عناصر ماتریس A می شود

```
;row_i := cntA / n
```

```
;col_i := cntA mod n
```

با استفاده از تقسیم و باقیمانده مشخص می شود بایت دریافتی مربوط به کدام سطر و ستون است

```
;bufA(row_i)(col_i*8+7 downto col_i*8) <= rx_byte
```

بایت در جای درست داخل بافر ذخیره می شود و هر عنصر ماتریس 8 بیت فضا اشغال می کند

```
;cntA <= cntA + 1
```

شمارنده افزایش پیدا می کند

```
if (cntA + 1) = totalA then
```

```
;stage <= READ_B
```

```
;end if
```

وقتی همه عناصر ماتریس A دریافت شد ماشین حالت وارد مرحله خواندن ماتریس B می شود

```
<= when READ_B
```

مشابه READ_A است اما داده ها داخل bufB ذخیره می شوند و پس از دریافت تمام عناصر ماتریس B ماشین حالت به

FINISH می رود

```
;row_i := cntB / m
```

```
;col_i := cntB mod m
```

```
;bufB(row_i)(col_i*8+7 downto col_i*8) <= rx_byte
```

```
;cntB <= cntB + 1
```

```
if (cntB + 1) = totalB then
```

```
;stage <= FINISH
```

```
;end if
```

```
<= when FINISH
```

```
;null
```

در اینجا هیچ دریافت جدیدی انجام نمی شود و سیستم آماده اجرای کانولوشن است

```
;end case
```

```
;end if
```

شرح بخش کانولوشن :

```
if (stage = FINISH) and (conv_done = '0') then
```

این شرط تضمین می‌کند که کانولوشن فقط زمانی اجرا شود که تمام داده‌ها دریافت شده باشد و قبلاً محاسبه انجام نشده باشد

اگر این شرط نباشد ممکن است محاسبه چند بار تکرار شود

```
;conv_done <= '1
```

```
;Cbuf <= (others => '0')
```

در اینجا سیستم اعلام می‌کند که وارد مرحله محاسبه شده است و بافر خروجی نیز پاک می‌شود تا داده‌ای از قبل نمانده باشد

```
;out_size := n - m + 1
```

```
;outN_sig <= std_logic_vector(to_unsigned(out_size, 4))
```

اندازه ماتریس خروجی در outN ذخیره می‌شود تا بیرون از ماژول هم قابل استفاده باشد

```
for r in 0 to out_size-1 loop
```

```
for c in 0 to out_size-1 loop
```

این دو حلقه روی تمام خانه‌های ماتریس خروجی حرکت می‌کنند که هر جفت r و c یک موقعیت در ماتریس حاصل است

```
;sum := (others => '0')
```

برای هر خانه خروجی جمع از صفر شروع می‌شود

این متغیر مجموع ضرب‌ها را نگه می‌دارد

```
for kr in 0 to m-1 loop
```

```
for kc in 0 to m-1 loop
```

این دو حلقه ماتریس B را روی A حرکت می‌دهند

```
;a_val := unsigned(bufA(r+kr)((c+kc)*8+7 downto (c+kc)*8))
```

```
;b_val := unsigned(bufB(kr)(kc*8+7 downto kc*8))
```

در این قسمت عنصر متناظر از A و B خوانده می‌شود

مکان A با جایابی r و c تعیین می‌شود و مکان B فقط به kr و kc وابسته است

هر عنصر 8 بیت است و به عدد unsigned تبدیل می‌شود

```
;sum := sum + resize(a_val * b_val, 32)
```

حاصل ضرب دو عنصر محاسبه می‌شود و نتیجه به 32 بیت تبدیل می‌شود سپس به جمع اضافه می‌شود

استفاده از 32 بیت باعث جلوگیری از سرریز می‌شود

```
;end loop
```

```
;end loop
```

```
;base := (r*10 + c) * 32
```

```
;Cbuf(base+31 downto base) <= std_logic_vector(sum)
```

نتیجه هر خانه در بافر خروجی ذخیره می‌شود

بافر خروجی 100 خانه 32 بیتی دارد یعنی نتایج در قالب یک ماتریس 10 در 10 ذخیره می‌شود

Base مکان دقیق هر نتیجه را در حافظه مشخص می‌کند

```
;end loop
```

```
;end loop
```

```
;end if
```

شرح بخش tx :

```
;signal cnt_tx : integer range 0 to DIV-1 := 0
```

این سیگنال یک شمارنده است که برای تولید زمان‌بندی baud rate استفاده می‌شود
در واقع کلاک 40 مگاهرتز را تقسیم می‌کند تا سرعت UART ساخته شود و هر بار که این شمارنده به مقدار DIV برسد یعنی
زمان ارسال یک بیت رسیده است

```
;signal tick_tx : std_logic := '0
```

این سیگنال یک پالس تک کلاکی است
وقتی cnt_tx به انتها برسد این سیگنال یک می‌شود و اعلام می‌کند که باید بیت بعدی ارسال شود

```
;signal tx_line : std_logic := '1
```

این خط خروجی واقعی UART است
وقتی بیکار است مقدار آن 1 است چون می‌دانیم که در UART حالت idle برابر 1 است

```
;signal tx_busy : std_logic := '0
```

این سیگنال مشخص می‌کند که فرستنده در حال ارسال است یا نه
اگر 1 باشد یعنی وسط ارسال یک فریم هستیم

```
;signal tx_shift : std_logic_vector(9 downto 0) := (others => '1')
```

این رجیستر شیفت شامل کل فریم UART است
یک start bit هشت بیت داده و یک stop bit که جمعاً 10 بیت می‌شود

```
;signal tx_bit_i : integer range 0 to 9 := 0
```

این متغیر اندیس بیت فعلی را نگه می‌دارد یعنی مشخص می‌کند کدام بیت از tx_shift الان باید روی خط قرار بگیرد

```
;signal tx_start : std_logic := '0
```

این سیگنال فرمان شروع ارسال یک بایت جدید است و ماشین حالت سطح بالا است که آن را یک می‌کند

```
;signal tx_byte : std_logic_vector(7 downto 0) := (others => '0')
```

بایستی است که باید ارسال شود اما قبل از شروع ارسال داخل این سیگنال قرار می‌گیرد

```
;signal send_started : std_logic := '0
```

```
;signal send_done : std_logic := '0
```

این دو سیگنال وضعیت ارسال کل داده‌ها را مشخص می‌کند یکی شروع عملیات ارسال و دیگری پایان کامل ارسال

```
) type txsend_state_t is
```

```
,TX_IDLE, TX_LOAD_WORD, TX_SEND_DIGIT
```

```
TX_SEND_CR, TX_SEND_LF, TX_NEXT_WORD
```

```
;
```

```
;signal txs_state : txsend_state_t := TX_IDLE
```

این ماشین حالت سطح بالا برای ارسال نتایج است که کنترل می‌کند چه چیزی و چه زمانی ارسال شود

```
;signal word_idx : integer range 0 to 99 := 0
```

این اندیس مشخص می‌کند کدام عدد از بافر خروجی در حال ارسال است

```
;signal dig_buf : std_logic_vector(79 downto 0) := (others => '0')
```

این بافر برای نگهداری رقم‌های ASCII عدد استفاده می‌شود

```
;signal dig_len : integer range 1 to 10 := 1
```

تعداد رقم‌های عدد فعلی را نگه می‌دارد

```
;signal dig_pos : integer range 0 to 9 := 0
```

مشخص می‌کند الان کدام رقم در حال ارسال است

```
process(clk40)
```

پروسس اول

```
begin
```

```
if rising_edge(clk40) then
```

```
;tick_tx <= '0
```

در هر سیکل اول tick صفر می‌شود

```
if cnt_tx = DIV-1 then
```

اگر شمارنده به انتها برسد یعنی زمان بیت جدید رسیده

```
;cnt_tx <= 0
;tick_tx <= '1
else
;cnt_tx <= cnt_tx + 1
;end if
if tx_start = '1' then
;tx_start <= '0
;end if
```

در فرمان شروع فقط یک سیکل فعال می ماند بعد صفر می شود

```
if tick_tx = '1' then
```

فقط در زمان tick اجازه ارسال بیت داریم

```
if tx_busy = '0' then
```

به شرط اینکه فرستنده بیکار باشد

```
;tx_line <= '1
```

خط در حالت idle نگه داشته می شود

```
if tx_start = '1' then
```

```
;tx_shift <= '1' & tx_byte & '0
```

اگر دستور ارسال آمده باشد فریم UART ساخته می شود

start bit برابر صفر

stop bit برابر یک

```
;tx_busy <= '1
```

```
;tx_bit_i <= 0
```

ارسال شروع می شود از بیت صفر

```
;end if
```

```
else
```

```
;tx_line <= tx_shift(tx_bit_i)
```

در حالت busy هر tick یک بیت ارسال می‌شود

```
if tx_bit_i = 9 then
```

اگر آخرین بیت ارسال شد

```
;tx_busy <= '0
```

ارسال تمام می‌شود

```
;tx_line <= '1
```

```
else
```

```
;tx_bit_i <= tx_bit_i + 1
```

میریم سراغ بیت بعدی

```
;end if
```

```
;end if
```

```
;end if
```

```
;end if
```

```
;end process
```

```
process(clk40)
```

پروسس دوم نتایج را ارسال می‌کند

```
;variable v : integer
```

این متغیر مقدار عددی خروجی کانولوشن را نگه می‌دارد یعنی همان عددی که قرار است به متن تبدیل شود

```
;variable tmp : integer
```

برای جدا کردن رقم‌ها استفاده می‌شود بدون اینکه مقدار اصلی v خراب شود

```
;variable digits : integer
```

تعداد رقم‌های عدد را نگه می‌دارد مثلاً اگر عدد 1234 باشد مقدار digits برابر 4 می‌شود

```
;variable i : integer
```

```
;variable b : std_logic_vector(7 downto 0)
```

هر رقم بعد از تبدیل به ASCII داخل این متغیر قرار می‌گیرد

```
;variable word32 : std_logic_vector(31 downto 0)
```

یک عدد 32 بیتی از بافر خروجی کانولوشن داخل این متغیر خوانده می‌شود

```
begin
```

```
if rising_edge(clk40) then
```

تمام این منطق روی لبه بالارونده کلاک اجرا می‌شود یعنی یک مدار سنکرون است

```
if conv_done = '1' then
```

```
;'send_started <= '1
```

```
;end if
```

وقتی کانولوشن تمام شود سیستم اجازه پیدا می‌کند نتایج را ارسال کند

```
case txs_state is
```

از اینجا کنترل کامل با ماشین حالت است که هر حالت یک مرحله از ارسال را انجام می‌دهد

```
<= when TX_IDLE
```

این حالت انتظار است سیستم کاری نمی‌کند تا وقتی ارسال شروع شود

```
if (send_started = '1') and (send_done = '0') then
```

اگر ارسال شروع شده و هنوز تمام نشده

```
;word_idx <= 0
```

از اولین عدد خروجی شروع کن

```
;txs_state <= TX_LOAD_WORD
```

می‌رود سراغ بارگذاری عدد

```
;end if
```

```
<= when TX_LOAD_WORD
```

```
if tx_busy = '0' then
```

در این حالت ابتدا بررسی می‌شود که فرستنده UART آزاد باشد
اگر هنوز در حال ارسال بایت قبلی باشد سیستم صبر می‌کند تا تداخل ایجاد نشود

```
;word32 := Cbuf(word_idx*32 + 31 downto word_idx*32)
```

در این خط یک عدد ۳۲ بیتی از حافظه خروجی کانولوشن خوانده می‌شود
word_idx مشخص می‌کند کدام خانه از نتایج باید ارسال شود

```
;v := to_integer(unsigned(word32))
```

عدد باینری به یک عدد صحیح تبدیل می‌شود چون برای تبدیل به رقم‌های ده‌دهی نیاز به عملیات تقسیم داریم

```
if v = 0 then
```

```
;dig_len <= 1
```

```
;"dig_buf(7 downto 0) <= x"30
```

اگر عدد صفر باشد یک حالت خاص در نظر گرفته می‌شود
طول عدد برابر ۱ است و فقط کاراکتر ASCII صفر در بافر ذخیره می‌شود

```
else
```

```
;tmp := v
```

```
;digits := 0
```

```
while tmp > 0 loop
```

```
;tmp := tmp / 10
```

```
;digits := digits + 1
```

```
;end loop
```

در غیر این صورت ابتدا تعداد رقم‌های عدد محاسبه می‌شود
با تقسیم متوالی بر ۱۰ مشخص می‌شود عدد چند رقمی است

```
;dig_len <= digits
```

تعداد رقم‌ها در متغیر طول ذخیره می‌شود تا سیستم بداند چند کاراکتر باید ارسال کند

```
for i in 0 to 9 loop
;"dig_buf(i*8+7 downto i*8) <= x"30
;end loop
```

کل بافر رقم‌ها با کاراکتر صفر پر می‌شود این کار باعث می‌شود داده‌های قدیمی در حافظه باقی نماند

```
;tmp := v
for i in 0 to digits-1 loop
;b := std_logic_vector(to_unsigned((tmp mod 10) + 48, 8))
;dig_buf((digits-1-i)*8+7 downto (digits-1-i)*8) <= b
;tmp := tmp / 10
;end loop
```

در این حلقه رقم‌های واقعی عدد استخراج می‌شوند و باقیمانده تقسیم بر ۱۰ رقم بعدی را می‌دهد سپس عدد ۴۸ اضافه می‌شود تا به کد ASCII تبدیل شود
رقم‌ها از راست به چپ داخل بافر قرار می‌گیرند

```
;end if
```

```
;dig_pos <= 0
;txs_state <= TX_SEND_DIGIT
;end if
```

موقعیت ارسال روی رقم اول تنظیم می‌شود و ماشین حالت وارد مرحله ارسال رقم‌ها می‌شود

```
<= when TX_SEND_DIGIT
if tx_busy = '0' then
```

در این حالت هر بار یک رقم ارسال می‌شود باز هم شرط آزاد بودن فرستنده بررسی می‌شود

```
;tx_byte <= dig_buf(dig_pos*8+7 downto dig_pos*8)
;'tx_start <= '1
```

رقم فعلی از بافر خوانده می شود و ارسال آن شروع می شود

```
if dig_pos = dig_len-1 then  
;txs_state <= TX_SEND_CR
```

اگر این رقم آخر باشد سیستم آماده رفتن به خط بعد می شود

```
else  
;dig_pos <= dig_pos + 1  
;end if  
;end if
```

در غیر این صورت اندیس جلو می رود تا رقم بعدی ارسال شود

```
<= when TX_SEND_CR  
if tx_busy = '0' then  
;"tx_byte <= x"0D  
;'tx_start <= '1  
;txs_state <= TX_SEND_LF  
;end if
```

در این مرحله کاراکتر CR ارسال می شود که باعث برگشتن به ابتدای خط در ترمینال می شود

```
<= when TX_SEND_LF  
if tx_busy = '0' then  
;"tx_byte <= x"0A  
;'tx_start <= '1  
;txs_state <= TX_NEXT_WORD  
;end if
```

اینجا کاراکتر LF ارسال می شود در نتیجه به خط بعدی می رود

```
<= when TX_NEXT_WORD  
if tx_busy = '0' then
```

در این مرحله بررسی می‌کند که عدد بعدی وجود دارد یا نه

```
if word_idx = 99 then
```

```
;'send_done <= '1
```

```
;txs_state <= TX_IDLE
```

اگر به آخرین نتیجه رسیده باشیم یعنی اینکه ارسال کامل شده و سیستم به حالت بیکار می‌رود

```
else
```

```
;word_idx <= word_idx + 1
```

```
;txs_state <= TX_LOAD_WORD
```

```
;end if
```

```
;end if
```

در غیر این صورت اندیس افزایش پیدا می‌کند

و سیستم برمی‌گردد تا عدد بعدی را آماده کند

```
;end case
```

```
;end if
```

```
;end process
```

از حسن توجه شما بی نهایت سپاسگزارم